Why don’t we use let/const for #x?

- Because #x is not a variable, it is a private class field. In JavaScript class syntax, fields are defined directly without let or const, and the # prefix makes them truly private at the language level.

#x is enforced by the JS engine itself.

- Not just naming convention
- Not just closure trick
- It is syntax-level private field

- JavaScript uses dynamic dispatch. The method call is resolved at runtime based on the actual object instance. Since the object is an instance of the child class, the overridden method in the child class is invoked even when the call originates from the parent constructor.

```
obj
 ↓
C.prototype   ← speak found here ✅
 ↓
B.prototype   ← never reached ❌
 ↓
A.prototype
```

- example

```
class Circle extends Shape {
    constructor() {
        this.r = 10; // ❌ ERROR - Must call super constructor before using 'this'
    }
}
```

- super() calls the parent class constructor and initializes the this object. Internally, it behaves similar to calling ParentConstructor.call(this), ensuring that the parent class properties are assigned to the same object instance.

If a child class has NO constructor, JavaScript automatically inserts one.

for example -

```
class B extends A {}
```

Is treated as

```
class B extends A {
    constructor(...args) {
        super(...args);
    }
}
```

- `Case 1` : No constructor → super() auto-called ✅
- `Case 2` : ou define constructor BUT don't call super ❌

In JS you must call super constructror before using 'this'

- Because in derived classes, this is NOT created automatically.

Because JS does NOT use classical inheritance like C++.

instead it uses prototype-based inheritance

- prototype (Property of a FUNCTION (or class)) vs **proto** (Property of an OBJECT (instance))
- Constructor.prototype → becomes → obj.**proto**

prototype is a property of constructor functions that defines properties shared by all instances, while **proto** is a property of an object that points to its internal prototype. When an object is created using new, its **proto** is set to the constructor's prototypes

```
let obj = {};
obj.__proto__ = Circle.prototype;

Shape.call(obj);
obj.r = 5;

return obj;
```

- This is minial simulation of new Circle()

Lets define meaning in JS internals -

- Let's convert it to real JS meaning

1. Create empty object

   ```
   let obj = {};
   ```

2. Set prototype
   obj.**proto** = Circle.prototype;
   👉 Now:
   obj behaves like instance of Circle

Prototype chain:
obj → Circle.prototype → Shape.prototype → Object.prototype

3. Call parent constructor
    - Shape.call(obj); // This equivalent to calling super() in child class, means this is behind the scene in JS

👉 This means:
Run Shape constructor with `this = obj`

So if:

```
class Shape {
    constructor() {
        this.type = "shape";
    }
}


Then:

obj = { type: "shape" }
```

4. Assign child property
   obj.r = 5;

Now:

obj = {
type: "shape",
r: 5
}

JavaScript has NO native interface keyword

Abstract Class Pattern (Closest to C++)

- JavaScript does not have a native interface keyword. However, we can simulate interfaces using abstract classes by throwing errors for unimplemented methods, or rely on duck typing. In production systems, TypeScript is commonly used to provide compile-time interface enforcement.

```
class Shape {
    constructor() {
        if (new.target === Shape) {
            throw new Error("Cannot instantiate abstract class");
        }
    }

    area() {
        throw new Error("Method 'area()' must be implemented");
    }
}
```

Child Class

```
class Circle extends Shape {
    constructor(r) {
        super();
        this.r = r; // this.r is NOT a variable → it’s an object property
    }

    area() {
        return Math.PI * this.r * this.r;
    }
}
```

new.target is a special meta-property in JavaScript that tells:

- "Which constructor was directly invoked using `new` "

class A {
constructor() {
console.log(new.target); //[class A]
}
}

new A();

Questions -

Q1

```
class A {
    speak() {
        console.log("A");
    }
}

class B extends A {
    speak() {
        super.speak();
        console.log("B");
    }
}

new B().speak();
```

O/P -

A

B

```
class A {
    constructor() {
        this.speak();
    }

    speak() {
        console.log("A");
    }
}

class B extends A {
    speak() {
        console.log("B");
    }
}

new B();
```

O/P -
B
IMP - Even inside parent constructor → child method is called
bCZ - method resolution happens at runtime (dynamic dispatch)

```
class A {
    speak() {
        console.log("A");
    }
}

class B extends A {
    speak = () => {
        console.log("B");
    }
}

let obj = new B();
obj.speak();
```

O/P

B

bcz - Instance property overrides prototype method.

```
class A {
    speak() {
        console.log("A");
    }
}

class B extends A {}

let obj = new B();

obj.speak = function () {
    console.log("Override");
};

obj.speak();
```

O/P

Override

Reason - instance > prototype

```
class A {
    speak() {
        console.log(this.name);
    }
}

class B extends A {
    constructor() {
        super();
        this.name = "B";
    }
}

const obj = new B();
const fn = obj.speak;
fn();
```

O/P - undefined

Fix with bind
const fn = obj.speak.bind(obj);
fn(); // B

```
class A {
    static hello() {
        console.log("A");
    }
}

class B extends A {}

B.hello(); // A
```

Reason - ✔ Static methods are inherited.

Static Override

```
class A {
static hello() {
console.log("A");
}
}

class B extends A {
static hello() {
console.log("B");
}
}

B.hello(); // B
```

```
class A {
name = "A";

  speak = () => {
      console.log(this.name);
  }

}

class B extends A {
name = "B";
}

new B().speak(); // B
```

reason - Arrow function captures this correctly. (lexical scoping)

```
class A {
speak() {
console.log("A");
```

```
}
}

class B extends A {}

const obj = new B();

A.prototype.speak = function () {
console.log("Modified A");
};

obj.speak(); // Modified A
```

reason - Modified A